Isolation levels for Sakila Dataset

Airon Matthew Bantillo
airon_bantillo@dlsu.edu.ph
De La Salle University
Manila, Philippines

Bullet Andre Fernandez
bullet_fernandez@dlsu.edu.ph
De La Salle University
Manila, Philippines

Charles Sebastian V. Infante
charles_infante@dlsu.edu.ph
De La Salle University
Manila, Philippines

Alec Marx Gabriel B. Nono
alec_nono@dlsu.edu.ph
De La Salle University
Manila, Philippines

Jean Rondel Ponce
jean_ponce@dlsu.edu.ph
De La Salle University
Manila, Philippines

1 Introduction

Modern relational database management systems (RDBMS) rely on concurrency controls to enable the simultaneous execution of multiple transactions while ensuring data integrity [4]. ANSI's SQL-92 standard [1] defines four transaction isolation levels to balance data consistency and control a system's susceptibility to the following three read anomalies: **Dirty Reads**, **Non-Repeatable Reads**, and **Phantom Reads** [2].

This study evaluates the operation behavior of MySQL's **InnoDB** storage engine under concurrent workload execution. A multi-session testing environment was constructed to observe Multi-Version Concurrency Control (MVCC) [6] and lock-based behavior across all four isolation levels [5]: **READ UNCOMMITTED**, **READ COMMITTED**, **REPEATABLE READ**, and **SERIALIZABLE**.

1.1 Relational Schema Design

The experimental schema is derived from MySQL's Sakila sample database [11]. The `customer` table is a simplified copy of `sakila.customer` reduced from nine columns to five (the `store_id` and `address_id` foreign keys and the `create_date` and `last_update` audit columns were dropped), and is populated with the real Sakila customers 1–5. Two five-column tables, `accounts` and `transactions`, model a payment/balance workload owned by those customers, following the banking examples used in the lecture slides [14]. All tables use the **InnoDB** engine, since MyISAM does not support transactions; the database is standalone, so Sakila itself does not need to be loaded.

The schema definition is implemented as follows (`sql/setup/00_schema.sql`):

Listing 1: DDL Schema Definition for Customer, Accounts, and Transactions

```
CREATE TABLE customer (
    customer_id SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
    first_name  VARCHAR(45) NOT NULL,
    last_name   VARCHAR(45) NOT NULL,
    email       VARCHAR(50) DEFAULT NULL,
    active      BOOLEAN NOT NULL DEFAULT TRUE,
    PRIMARY KEY (customer_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE accounts (
    account_id  INT PRIMARY KEY AUTO_INCREMENT,
    customer_id SMALLINT UNSIGNED NOT NULL,
    balance     DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
    status      VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
    updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON
     UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (customer_id) REFERENCES
     customer(customer_id)
) ENGINE=InnoDB;

CREATE TABLE transactions (
    transaction_id INT PRIMARY KEY AUTO_INCREMENT,
    account_id     INT NOT NULL,
    type           VARCHAR(20) NOT NULL, -- 'DEPOSIT',
     'WITHDRAWAL', 'TRANSFER'
    amount         DECIMAL(10, 2) NOT NULL,
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (account_id) REFERENCES accounts(account_id)
) ENGINE=InnoDB;
```

1.2 Initial Seed Data

The environment was initialized with three core account entities to provide baseline states for concurrent query execution (`sql/setup/01_seed.sql`). Before every test case the runner truncates `accounts` and `transactions` and re-applies this seed.

Accounts Initial State

account_id	customer_id	balance	status
101	1 (MARY SMITH)	1000.00	ACTIVE
102	2 (PATRICIA JOHNSON)	500.00	ACTIVE
103	3 (LINDA WILLIAMS)	2500.00	ACTIVE

Transactions Initial State

account_id	type	amount
101	DEPOSIT	1000.00
102	DEPOSIT	500.00
103	DEPOSIT	2500.00

2 Experiments

To evaluate concurrency control across the four ANSI SQL-92 isolation levels (**READ UNCOMMITTED**, **READ COMMITTED**, **REPEATABLE READ**, and **SERIALIZABLE**), a multi-session experimental framework was implemented. The test harness employs

two concurrent database connections managed by a multithreaded Python harness with barrier synchronization (`threading.Barrier`) [12, 13], enforcing precise statement interleaving across dual sessions via controlled `DO SLEEP()` delays.

A total of 12 baseline experiments (4 isolation levels×3 read anomalies) were systematically executed and logged. Prior to every individual test case, the database state was reset and re-seeded using automated DDL/DML scripts to ensure deterministic initial conditions.

2.1 Experimental SQL Transactions

The transactional workloads were designed to isolate the three classical read phenomena. In accordance with SQL coding conventions, transactional commands, query targets, and timing delays are presented below.

2.1.1 Dirty Read Workload. A transaction updates Account 101 without committing, while a concurrent transaction reads the uncommitted balance before a rollback occurs.

Real-World Scenario. Session 1 models a customer withdrawing $300.00 at an ATM: the balance is debited first, but the machine then encounters a cash-dispensing fault, so the withdrawal is cancelled and rolled back. Session 2 models a bank teller checking the same customer's balance while the withdrawal is still in progress, as when assessing whether a large payment can be approved, and rechecking it moments later. If the teller sees $700.00, the decision is based on a balance that never officially existed, since the customer's funds remained $1000.00 throughout.

Listing 2: Session 1 (Writer) Transaction for Dirty Read

```
-- Session 1: Writer
SET SESSION TRANSACTION ISOLATION LEVEL <LEVEL>;
START TRANSACTION;
UPDATE accounts SET balance = balance - 300.00 WHERE
     account_id = 101;
SELECT 'W after update' AS step, balance FROM accounts
     WHERE account_id = 101;
DO SLEEP(6); -- Hold uncommitted change
ROLLBACK;    -- Reverts update back to 1000.00
SELECT 'W after rollback' AS step, balance FROM accounts
     WHERE account_id = 101;
```

Listing 3: Session 2 (Reader) Transaction for Dirty Read

```
-- Session 2: Reader
SET SESSION TRANSACTION ISOLATION LEVEL <LEVEL>;
DO SLEEP(2); -- Wait for Session 1's UPDATE
START TRANSACTION;
SELECT 'R1' AS step, balance FROM accounts WHERE account_id
     = 101;
DO SLEEP(6); -- Wait for Session 1's ROLLBACK
SELECT 'R2' AS step, balance FROM accounts WHERE account_id
     = 101;
COMMIT;
```

2.1.2 Non-Repeatable Read Workload. Session 1 performs an initial read on Account 101, pauses, and re-reads the row. Concurrently, Session 2 modifies and commits the balance during the pause.

Real-World Scenario. Session 1 models a statement-generation job that reads an account's balance for the summary section and reads it again when producing the closing total of the same document. Session 2 models a $200.00 payroll deposit that is posted and committed between those two reads. Although the deposit is legitimate, a statement showing $1000.00 in one section and $1200.00 in another is internally inconsistent because a single unit of work observed two different committed values.

Listing 4: Session 1 (Reader) Transaction for Non-Repeatable Read

```
-- Session 1: Reader
SET SESSION TRANSACTION ISOLATION LEVEL <LEVEL>;
START TRANSACTION;
SELECT 'R1' AS step, balance FROM accounts WHERE account_id
     = 101;
DO SLEEP(6); -- Pause while Session 2 updates and commits
SELECT 'R2' AS step, balance FROM accounts WHERE account_id
     = 101;
COMMIT;
```

Listing 5: Session 2 (Writer) Transaction for Non-Repeatable Read

```
-- Session 2: Writer
SET SESSION TRANSACTION ISOLATION LEVEL <LEVEL>;
DO SLEEP(2); -- Wait for Session 1's first read
START TRANSACTION;
UPDATE accounts SET balance = balance + 200.00 WHERE
     account_id = 101;
COMMIT;
SELECT 'W committed' AS step, balance FROM accounts WHERE
     account_id = 101;
```

2.1.3 Phantom Read Workload. Session 1 executes a range query on accounts with `balance >= 800.00`, pauses, and re-executes the query. Session 2 concurrently inserts a new qualifying row (Account 104 with balance $1500.00). In Session 1, an update statement is also executed to evaluate InnoDB current-read behavior under REPEATABLE READ.

Real-World Scenario. Session 1 models a compliance audit that lists all accounts holding at least $800.00, re-runs the listing to confirm it, and then flags every qualifying account for review. Session 2 models a branch opening a new account with an initial deposit of $1500.00 while the audit is running. The new account may appear as an unexpected extra row between the two listings; under REPEATABLE READ, it can remain hidden from both listings yet still be flagged by the update, leaving the audit with a flagged account that it never listed.

Listing 6: Session 1 (Reader) Transaction for Phantom Read

```
-- Session 1: Reader
SET SESSION TRANSACTION ISOLATION LEVEL <LEVEL>;
SET SQL_SAFE_UPDATES = 0;
START TRANSACTION;
SELECT 'P1' AS step, account_id, balance FROM accounts
     WHERE balance >= 800.00 ORDER BY account_id;
DO SLEEP(6); -- Pause while Session 2 inserts row 104
SELECT 'P2' AS step, account_id, balance FROM accounts
     WHERE balance >= 800.00 ORDER BY account_id;
UPDATE accounts SET status = 'FLAGGED' WHERE balance >=
     800.00;
```

```
SELECT 'P3' AS step, account_id, balance FROM accounts
     WHERE balance >= 800.00 ORDER BY account_id;
COMMIT;
```

Listing 7: Session 2 (Inserter) Transaction for Phantom Read

```
-- Session 2: Inserter
SET SESSION TRANSACTION ISOLATION LEVEL <LEVEL>;
DO SLEEP(2); -- Wait for Session 1's first range read
START TRANSACTION;
INSERT INTO accounts (account_id, customer_id, balance,
     status) VALUES (104, 4, 1500.00, 'ACTIVE');
COMMIT;
```

2.2 Summary of Baseline Results

Table 1 summarizes the empirical findings across all 12 baseline test cases, comparing ANSI-92 expectations with observed InnoDB behavior.

2.3 Two-Column Interleaved Timelines and Empirical Discussion

The empirical execution results for all 12 baseline configurations are summarized in Table 1. To preserve readability in the main text while providing complete empirical transparency, the full two-column interleaved execution traces (t_x) across all four isolation levels are grouped into three consolidated configuration tables located in the Appendix:

- **Dirty Read Workload (Cases 1–4):** Documented in Appendix Table 5 (with a manual Workbench capture of Case 1 shown in Figure 1), illustrating the visibility of uncommitted writes under READ UNCOMMITTED, undo-log pre-image reconstruction under READ COMMITTED and REPEATABLE READ, and Two-Phase Locking contention (4.0s lock block) under SERIALIZABLE.
- **Non-Repeatable Read Workload (Cases 5–8):** Documented in Appendix Table 6, contrasting statement-level Read Views (fresh snapshot per query under READ COMMITTED) against transaction-level Read View reuse under REPEATABLE READ, and Shared-Lock blocking under SERIALIZABLE.
- **Phantom Read Workload (Cases 9–12):** Documented in Appendix Table 7, demonstrating the classic phantom anomaly in READ UNCOMMITTED and READ COMMITTED, the snapshot hiding and write-only phantom manifestation in REPEATABLE READ, and index gap lock prevention in SERIALIZABLE.

2.3.1 Read View Mechanics: READ COMMITTED vs. REPEATABLE READ. The difference between READ COMMITTED and REPEATABLE READ for non-repeatable reads stems from allocation policies in Multi-Version Concurrency Control (MVCC) Read View. Under READ COMMITTED, InnoDB generates statement-level Read Views at the execution of each individual `SELECT` [5, 6]. As session 2 updates to \$1200.00 at $t = 2.0$s, Session 1's query at $t = 6.0$s generates a new Read View that incorporates this committed state, resulting in a non-repeatable read ($R_1 \neq R_2$). In contrast, REPEATABLE READ establishes a single, transaction-level Read View upon the execution of a transaction's first read ($t = 0.0$s), maintaining it until termination. Even if Session 2 commits changes concurrently, Session 1's subsequent read traverses the undo log history to reconstruct the committed pre-image of \$1000.00, successfully preserving read repeatability.

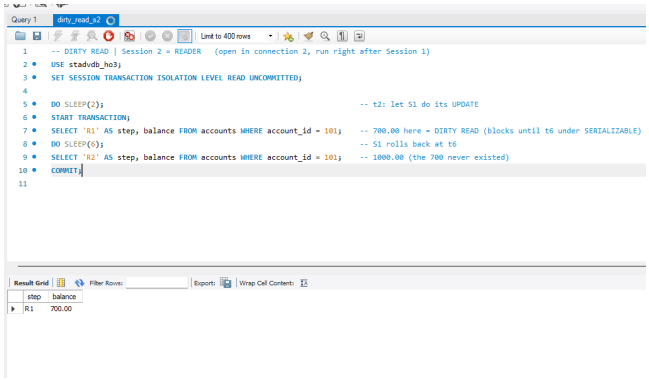


Figure 1: Manual validation of Dirty Read in MySQL Workbench under READ UNCOMMITTED: Result 1 (R_1) displays uncommitted balance (700.00)

2.3.2 The Phantom Phenomenon under REPEATABLE READ. The "Write-Only Phantom" under REPEATABLE READ highlights a distinction between InnoDB's MVCC Snapshot Reads and Current Reads. According to ANSI SQL-92's specifications [1, 2], REPEATABLE READ permits phantom rows on re-queries. However, InnoDB mitigates standard read phantoms by utilizing consistent read snapshots; as observed in query P_2 at $t = 6.0$s, Session 1's plain `SELECT` continues to return only the original two accounts ([101, 103]), completely concealing Session 2's committed insert of Account 104. However, when Session 1 subsequently executes a range modification (`UPDATE accounts SET status = 'FLAGGED' WHERE balance >= 800.00`), the database engine switches to a Current Read to ensure data integrity [6, 7]. The `UPDATE` scans the current clustered index, matches the committed row 104, and updates it. Crucially, this operation rewrites row 104's header metadata (`DB_TRX_ID`) with Session 1's own transaction ID. Because MVCC rules dictate that a transaction can always observe its own writes, the subsequent query P_3 reveals Account 104 alongside accounts 101 and 103. The phantom is therefore "write-only", as it cannot be triggered through non-locking reads alone but is unmasked once a transaction modifies a phantom record.

2.3.3 Strict Two-Phase Locking and Empirical Lock Contention. The 4.0-second execution delay observed in the SERIALIZABLE logs validates Strict Two-Phase Locking (SS2PL) and lock compatibility enforcement [3, 4]. In SERIALIZABLE mode, as non-locking MVCC reads are disabled [5], all `SELECT` statements acquire Shared Locks (S-locks), while `UPDATE` statements acquire Exclusive Locks (X-locks). In the Dirty Read experiment (Case 4), Session 1 enters its Growing Phase at $t = 0.0$s by acquiring an X-lock on Account 101 and holds it across its 6-second sleep. When Session 2 issues a read at $t = 2.0$s requesting an S-lock, InnoDB blocks the request due to S/X lock conflict. Under Strict 2PL, Session 1 only enters its

Table 1: Comprehensive 12-Case Baseline Concurrency Matrix on MySQL InnoDB

Case	Anomaly	Isolation Level	ANSI-92	InnoDB Observed	Match	Observed Behavioral Note
1	Dirty Read	READ UNCOMMITTED	Allowed	**ANOMALY**	Yes	S2 reads uncommitted balance ($700.00) at t = 2.0s prior to S1 rollback.
2	Dirty Read	READ COMMITTED	Prevented	None	Yes	S2 reads committed pre-image ($1000.00) from undo log.
3	Dirty Read	REPEATABLE READ	Prevented	None	Yes	S2 reads from consistent snapshot ($1000.00).
4	Dirty Read	SERIALIZABLE	Prevented	None	Yes	S2 read is blocked for 4.0s on S1's Exclusive Lock until rollback.
5	Non-Repeatable Read	READ UNCOMMITTED	Allowed	**ANOMALY**	Yes	S1 reads $1000.00 at t = 0s and $1200.00 at t = 6s ($R_1 \neq R_2$).
6	Non-Repeatable Read	READ COMMITTED	Allowed	**ANOMALY**	Yes	Second query establishes fresh Read View, reflecting committed $1200.00.
7	Non-Repeatable Read	REPEATABLE READ	Prevented	None	Yes	Transaction-level Read View preserves $1000.00 pre-image.
8	Non-Repeatable Read	SERIALIZABLE	Prevented	None	Yes	S1's Shared Lock blocks S2's UPDATE for 4.0s until S1 commits.
9	Phantom Read	READ UNCOMMITTED	Allowed	**ANOMALY**	Yes	Newly inserted Account 104 appears on second range query (P_2: 3 rows).
10	Phantom Read	READ COMMITTED	Allowed	**ANOMALY**	Yes	Fresh statement Read View reveals newly committed Account 104.
11	Phantom Read	REPEATABLE READ	Allowed	**Write-Only**	Yes	Snapshot hides Account 104 in P_2, but range UPDATE touches 104, revealing it in P_3.
12	Phantom Read	SERIALIZABLE	Prevented	None	Yes	S1's Next-Key Locks block S2's INSERT for 4.0s until S1 commits.

Shrinking Phase upon issuing `ROLLBACK` at t = 6.0s, releasing all acquired locks. Session 2 is immediately granted the S-lock, logging an exact lock wait duration of 4.0s (6.0s − 2.0s). An identical 4.0s block occurs in Case 8, where Session 1's S-lock blocks Session 2's X-lock update until Session 1 commits.

2.4 Explicit Locking Experiments

To evaluate lock-based concurrency control as an alternative to elevating the isolation level of an entire transaction, six explicit locking configurations were tested, covering all four isolation levels. Cases 1–4 keep an isolation level at which the target anomaly can occur and add an explicit lock [7, 10]. Because SERIALIZABLE already prevents all three anomalies (Cases 4, 8, and 12), Cases 5 and 6 instead hold the anomaly-prevention goal fixed at SERIALIZABLE and vary only the lock *granularity*, so that its concurrency cost can be measured directly:

(1) **Lock Case 1 (Preventing Dirty Reads under READ UNCOMMITTED):** Session 2 reads Account 101 using `SELECT ... FOR UPDATE`, so it must wait for Session 1's exclusive row lock instead of reading the uncommitted balance.
(2) **Lock Case 2 (Preventing Non-Repeatable Reads under READ COMMITTED):** Session 1 reads Account 101 using `SELECT ... FOR SHARE`, holding a shared row lock that blocks Session 2's `UPDATE` until Session 1 commits.
(3) **Lock Case 3 (Preventing Phantom Reads under REPEATABLE READ):** Session 1 performs a locking range read (`SELECT ... WHERE balance >= 800.00 FOR UPDATE`), which acquires next-key locks [9] that block Session 2's `INSERT`.
(4) **Lock Case 4 (Preventing Phantom Reads under READ COMMITTED):** Session 1 acquires a table-level lock using `LOCK TABLES accounts READ`, which blocks Session 2's `INSERT` until `UNLOCK TABLES`.
(5) **Lock Case 5 (Row-Level Lock under SERIALIZABLE):** Session 1 reads Account 101 twice with `SELECT ... FOR UPDATE`, while Session 2 updates the *unrelated* Account 102.
(6) **Lock Case 6 (Table-Level Lock under SERIALIZABLE):** The same workload as Case 5, except Session 1 protects its reads with `LOCK TABLES accounts WRITE`.

Table 2 maps each isolation level to the lock experiments run at that level.

Table 2: Lock Experiment Coverage per Isolation Level

Isolation Level	Lock Cases
READ UNCOMMITTED	1 (row, `FOR UPDATE`)
READ COMMITTED	2 (row, `FOR SHARE`); 4 (table, `READ`)
REPEATABLE READ	3 (row/next-key, `FOR UPDATE`)
SERIALIZABLE	5 (row, `FOR UPDATE`); 6 (table, `WRITE`)

2.4.1 Lock Experiment Transactions. The listings below show the statements that differ from the corresponding baseline workloads (`sql/cases/lock_case<n>_s<1|2>.sql`); the `SET SESSION TRANSACTION ISOLATION LEVEL` line of each session is fixed to the level in Table 2. The full two-column timelines are given in Appendix Table 8.

Listing 8: Lock Case 1: Session 2 Locking Read (READ UNCOMMITTED)

```
-- Session 1 is unchanged from the Dirty Read writer
DO SLEEP(2);
START TRANSACTION;
-- Blocks until Session 1's ROLLBACK releases its X-lock
SELECT 'R1' AS step, balance FROM accounts WHERE account_id
     = 101 FOR UPDATE;
DO SLEEP(2);
SELECT 'R2' AS step, balance FROM accounts WHERE account_id
     = 101 FOR UPDATE;
COMMIT;
```

Listing 9: Lock Case 2: Session 1 Shared Locking Read (READ COMMITTED)

```
-- Session 2 is unchanged from the Non-Repeatable Read
     writer
START TRANSACTION;
SELECT 'R1' AS step, balance FROM accounts WHERE account_id
     = 101 FOR SHARE;
DO SLEEP(6); -- Session 2's UPDATE waits on the S-lock
SELECT 'R2' AS step, balance FROM accounts WHERE account_id
     = 101 FOR SHARE;
COMMIT;
```

Listing 10: Lock Case 3: Session 1 Locking Range Read (REPEATABLE READ)

```
-- Session 2 is unchanged from the Phantom Read inserter
START TRANSACTION;
-- balance is unindexed: full PRIMARY KEY scan locks every
     row and gap
SELECT 'P1' AS step, account_id, balance FROM accounts
     WHERE balance >= 800.00 FOR UPDATE;
DO SLEEP(6);
SELECT 'P2' AS step, account_id, balance FROM accounts
     WHERE balance >= 800.00 FOR UPDATE;
COMMIT;
```

Listing 11: Lock Case 4: Session 1 Explicit Table Lock (READ COMMITTED)

```
-- Session 2 is unchanged from the Phantom Read inserter
-- No START TRANSACTION: it would implicitly release the
     table lock
LOCK TABLES accounts READ;
SELECT 'P1' AS step, account_id, balance FROM accounts
     WHERE balance >= 800.00 ORDER BY account_id;
DO SLEEP(6);
SELECT 'P2' AS step, account_id, balance FROM accounts
     WHERE balance >= 800.00 ORDER BY account_id;
UNLOCK TABLES;
```

Listing 12: Lock Cases 5 and 6: Row Lock vs. Table Lock (SERIALIZABLE)

```
-- Session 1, Case 5 (row lock)       -- Session 1, Case 6
     (table lock)
START TRANSACTION;                    -- LOCK TABLES
     accounts WRITE;
SELECT 'R1' AS step, balance FROM accounts WHERE account_id
     = 101 FOR UPDATE;
DO SLEEP(6);
SELECT 'R2' AS step, balance FROM accounts WHERE account_id
     = 101 FOR UPDATE;
COMMIT;                               -- UNLOCK TABLES;

-- Session 2 (identical in both cases): writes an UNRELATED
     row
DO SLEEP(2);
START TRANSACTION;
UPDATE accounts SET balance = balance + 50.00 WHERE
     account_id = 102;
COMMIT;
```

2.4.2 Lock Experiment Results. Table 3 summarizes the results. In Cases 1–4, the anomaly observed at the corresponding baseline isolation level was prevented, at the cost of a 4.0s wait in Session 2.

Supplementary runs clarified the scope of the two phantom-prevention methods. Because `balance` is not indexed, the locking read in Lock Case 3 scans the entire primary key and locks every row and gap, not only the range `balance >= 800.00`. An `INSERT` with a balance of $100.00, outside the queried range, was likewise blocked for 4.0s; after an index was added on `balance`, that insert proceeded immediately while a $1500.00 insert remained blocked. The same locking read did not prevent the phantom under READ COMMITTED, where InnoDB does not use gap locks for this search: the `INSERT` proceeded immediately and Account 104 appeared in P2.

Lock Case 4 prevents this READ COMMITTED phantom at the coarser granularity of the whole table, so all concurrent writes to `accounts` are blocked while the lock is held. Its script deliberately omits `START TRANSACTION`, which implicitly releases table locks: when the statement was added after `LOCK TABLES`, the `INSERT` proceeded immediately and the phantom returned.

Lock Cases 5 and 6 isolate the cost of that coarser granularity. Both keep Session 1's two reads of Account 101 consistent, so neither changes *correctness* relative to SERIALIZABLE. They differ in what else they block: the row lock in Case 5 covers only the Account 101 index record, so Session 2's write to Account 102 does not conflict (logging 0.0s lock wait), whereas the table lock in Case 6 blocks every write to `accounts`, forcing Session 2 to endure a 4.0s collateral wait for a row Session 1 never reads.

2.4.3 Performance Trade-offs of Lock Granularity. The 4.0s waits above are dictated by the `DO SLEEP` schedule and therefore show *that* a lock blocks, not how much throughput it costs. To quantify the trade-off, a separate benchmark (`runner/benchmark_locks.py`) runs N concurrent worker threads, each executing K short transactions that read a row, hold their lock for h seconds of simulated work, and then update the row, all under SERIALIZABLE. Three modes are compared:

- **row**: `SELECT ... FOR UPDATE`, each worker on its own row (disjoint work, fine-grained lock).
- **row-hot**: `SELECT ... FOR UPDATE`, all workers on the same row (contended work, fine-grained lock).
- **table**: `LOCK TABLES accounts WRITE`, each worker on its own row (disjoint work, coarse lock).

Table 3: Explicit Locking Results on MySQL InnoDB

Case	Anomaly	Isolation Level	Explicit Lock	S2 Wait	Observed Result
1	Dirty Read	READ UNCOMMITTED	`SELECT ... FOR UPDATE`	4.0s	S2 reads $1000.00 after S1's rollback at $t = 6.0$s instead of the uncommitted $700.00.
2	Non-Repeatable Read	READ COMMITTED	`SELECT ... FOR SHARE`	4.0s	S1 reads $1000.00 in both R1 and R2; S2's update commits $1200.00 only after S1 commits.
3	Phantom Read	REPEATABLE READ	`SELECT ... FOR UPDATE`	4.0s	P1 and P2 both return Accounts 101 and 103; Account 104 is inserted only after S1 commits.
4	Phantom Read	READ COMMITTED	`LOCK TABLES ... READ`	4.0s	P1 and P2 both return Accounts 101 and 103; Account 104 is inserted only after `UNLOCK TABLES`.
5	Non-Repeatable Read	SERIALIZABLE	`SELECT ... FOR UPDATE`	0.0s	S1 reads $1000.00 in R1 and R2; S2's update of Account 102 completes immediately (0.0s wait) without collateral blocking.
6	Non-Repeatable Read	SERIALIZABLE	`LOCK TABLES ... WRITE`	4.0s	S1 reads $1000.00 in R1 and R2; S2's update of Account 102 blocks for 4.0s until `UNLOCK TABLES` due to coarse granularity.

If locks only serialize conflicting work, the ideal runtime is $K \cdot h$ for **row** and $N \cdot K \cdot h$ for the other two modes. Table 4 reports the measured values.

Table 4: Lock Granularity Benchmark (SERIALIZABLE, $N = 4$, $K = 5$, $h = 0.2$s)

Mode	Ideal	Runtime	txn/s	Avg latency
row	1.00s	1.18s	16.93	0.22s
row-hot	4.00s	4.39s	4.56	0.80s
table	4.00s	4.44s	4.51	0.81s

The expected pattern is that **table** performs about as badly as **row-hot**: an explicit table lock makes four workers with disjoint rows queue up as if they were all contending for a single row, cutting throughput by roughly a factor of N relative to **row**. Row-level locks recover that concurrency only when the work is actually disjoint and the lookup uses an index; Lock Case 3 showed that a locking read on the unindexed `balance` column degrades into locking the whole table anyway. The trade-off is therefore not "row locks are fast, table locks are slow" but rather that lock scope should match the data a transaction actually depends on: table locks are simple and deadlock-free (all locks are acquired up front) but sacrifice concurrency, whereas row and next-key locks preserve concurrency at the cost of index design discipline and the possibility of deadlocks and `innodb_lock_wait_timeout` errors [8].

3 Insights

3.1 Isolation Is a Dial, Not a Switch

The 12 baseline cases show that no single isolation level is "correct"; each level removes one more class of anomaly by doing more work per read. READ UNCOMMITTED reads whatever is in the buffer and exposes the $700.00 balance that was later rolled back. READ COMMITTED pays for a fresh Read View per statement and removes dirty reads, yet still lets a statement job see $1000.00 and then $1200.00 within one transaction. REPEATABLE READ keeps one Read View per transaction and, on InnoDB, even hides phantom inserts from plain `SELECT`s. SERIALIZABLE converts every read into a locking read and is the only level that prevented all three anomalies outright, but it did so by making Session 2 wait 4.0s in every SERIALIZABLE case. The DBMS exposes the isolation level because only the application knows which of these costs is acceptable: a dashboard showing approximate totals can tolerate READ COMMITTED, whereas a funds transfer that reads a balance and then writes based on it cannot.

3.2 The Standard Is Not the Implementation

ANSI SQL-92 defines isolation levels by which phenomena they *permit*, and engines are free to prevent more [2]. InnoDB's REPEATABLE READ prevented the classic phantom in P_2, which the standard allows, yet still produced the "write-only" phantom in P_3, where an `UPDATE` modified and then revealed Account 104, a row the transaction had never seen. A developer who assumes "REPEATABLE READ means no phantoms" on InnoDB would write an audit that flags an account it never listed; a developer who assumes "REPEATABLE READ allows phantoms" would add unnecessary locking. Application code therefore has to be validated against the specific engine and version, not only against the textbook isolation matrix.

3.3 Targeted Locks Beat Blanket Isolation

The lock experiments show that an anomaly can be removed for one critical statement without raising the isolation level of the entire session. Lock Case 1 prevented a dirty read at READ UNCOMMITTED, and Lock Case 2 prevented a non-repeatable read at READ COMMITTED, by adding `FOR UPDATE` or `FOR SHARE` to a single `SELECT`. This lets an application run most of its traffic at a cheap level (MySQL's default REPEATABLE READ or the commonly used READ COMMITTED) and pay for locking only on the read-then-write paths that need it, such as checking a balance before a withdrawal. The cost is that correctness now depends on every developer remembering to write the locking read; isolation levels are declarative and enforced by the engine, whereas explicit locks are imperative and easy to omit.

3.4 Lock Scope Must Match Data Dependencies

Lock Case 3 blocked an insert with a balance of $100.00, outside the audited range, because the unindexed predicate forced InnoDB to lock every row and gap, and the same locking read failed to prevent the phantom at READ COMMITTED, where gap locks are not used. Lock Cases 4 and 6 achieved correctness with table locks but blocked writes to unrelated rows, and the benchmark in Table 4 quantifies how that collateral blocking turns disjoint workers into a single queue. Choosing a locking strategy is therefore a schema-design decision as much as a transaction-design decision: indexes determine how much a locking read actually locks, and the isolation level determines whether gap locks exist at all.

3.5 Impact on Application Development

In practice, these findings translate into concrete development rules. First, choose the isolation level per transaction, not per database: `SET SESSION TRANSACTION ISOLATION LEVEL` or `SET TRANSACTION` lets a reporting query and a payment run side by side with different guarantees. Second, make transactions short; every SERIALIZABLE and locking case in this study waited exactly as long as the other transaction held its locks, so the `DO SLEEP` calls stand in for slow application logic, network round-trips, or user think-time inside a transaction. Third, expect and handle lock wait timeouts and deadlocks (MySQL errors 1205 and 1213) with retry logic, since stronger isolation turns silent anomalies into explicit errors. Finally, test concurrency behavior empirically, as this study did with the two-session runner, because the interaction between MVCC snapshots, current reads, and gap locks is subtle enough that reasoning from documentation alone was insufficient to predict the REPEATABLE READ and READ COMMITTED locking results.

A Baseline Concurrency Execution Timelines

This appendix details the physical query execution traces across dual database sessions (Session 1 and Session 2) captured by the automated concurrency runner. All four ANSI SQL-92 isolation levels (**READ UNCOMMITTED**, **READ COMMITTED**, **REPEATABLE READ**, and **SERIALIZABLE**) are evaluated across the three concurrency problem configurations. Elapsed execution timestamps (t_x) denote seconds from initial barrier synchronization ($t = 0.0$s).

A.1 Dirty Read Baseline Timelines (Cases 1–4)

Table 5 consolidates the execution traces for the Dirty Read workload across all four isolation levels.

A.2 Non-Repeatable Read Baseline Timelines (Cases 5–8)

Table 6 consolidates the execution traces for the Non-Repeatable Read workload across all four isolation levels.

A.3 Phantom Read Baseline Timelines (Cases 9–12)

Table 7 consolidates the execution traces for the Phantom Read workload across all four isolation levels.

B Lock Experiment Execution Timelines

Table 8 gives the two-column interleaved timelines for the six lock experiments, using the same timestamp convention as Appendix A.

References

[1] American National Standards Institute. 1992. *ANSI X3.135-1992: Information Systems — Database Language — SQL.* ANSI, New York, NY.

[2] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O'Neil, and Patrick O'Neil. 1995. A Critique of ANSI SQL Isolation Levels. In *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data.* 1–10. https://doi.org/10.1145/223784.223785

[3] Jim Gray and Andreas Reuter. 1992. *Transaction Processing: Concepts and Techniques.* Morgan Kaufmann.

[4] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. 2020. *Database System Concepts* (7th ed.). McGraw-Hill.

[5] Oracle Corporation. 2025. Transaction Isolation Levels. In *MySQL 9.4 Reference Manual.* https://dev.mysql.com/doc/refman/9.4/en/innodb-transaction-isolation-levels.html

[6] Oracle Corporation. 2025. Consistent Nonlocking Reads. In *MySQL 9.4 Reference Manual.* https://dev.mysql.com/doc/refman/9.4/en/innodb-consistent-read.html

[7] Oracle Corporation. 2025. Locking Reads. In *MySQL 9.4 Reference Manual.* https://dev.mysql.com/doc/refman/9.4/en/innodb-locking-reads.html

[8] Oracle Corporation. 2025. InnoDB Locking. In *MySQL 9.4 Reference Manual.* https://dev.mysql.com/doc/refman/9.4/en/innodb-locking.html

[9] Oracle Corporation. 2025. Phantom Rows. In *MySQL 9.4 Reference Manual.* https://dev.mysql.com/doc/refman/9.4/en/innodb-next-key-locking.html

[10] Oracle Corporation. 2025. LOCK TABLES and UNLOCK TABLES Statements. In *MySQL 9.4 Reference Manual.* https://dev.mysql.com/doc/refman/9.4/en/lock-tables.html

[11] Oracle Corporation. 2025. *Sakila Sample Database.* https://dev.mysql.com/doc/sakila/en/

[12] Python Software Foundation. 2025. threading — Thread-based parallelism. In *The Python Standard Library.* https://docs.python.org/3/library/threading.html

[13] PyMySQL Contributors. 2025. *PyMySQL: Pure Python MySQL Client.* https://github.com/PyMySQL/PyMySQL

[14] De La Salle University, College of Computer Studies. 2026. STADVDB Lecture Notes: Transaction Management, Concurrency Control, and Isolation Levels. Course lecture slides.

C Declaration of Generative AI Use

During the conduct of this hands-on activity, we used **Claude** (Anthropic) and **Gemini** (Google) to assist with the following tasks:

- **Log Processing and Timeline Tabulation:** Ingesting raw, timestamped dual-session execution logs generated by the multithreaded test harness and transforming them into structured, two-column interleaved LaTeX timeline tables for visual empirical analysis.
- **Document Preparation and Code Formatting:** Formatting SQL transactions, schema listings, and benchmark result matrices into standard ACM SIGCONF / CCS LaTeX markup and bibtex citation structures.
- **Critical Review of Empirical Explanations:** Reviewing our written analyses to identify lapses, ambiguities, or gaps in explaining the observed phenomenon under MySQL InnoDB (e.g., contrasting MVCC snapshot reads against current-read metadata changes in REPEATABLE READ write-only phantoms).
- **Conceptual Understanding and Alternative Concurrency Formulations:** Deepening conceptual comprehension of transaction isolation levels, ANSI SQL-92 anomalies, and locking mechanisms, exploring alternative query patterns to systematically replicate anomalies, and critiquing experimental reasoning without supplanting our own critical analysis.

- **Analysis of Lock Performance Trade-offs:** Formulating discussion and benchmarking approaches to evaluate throughput and latency costs associated with lock scope (row-level locking reads versus table-level locks).

Statement of Accountability. By submitting this report, we explicitly affirm the following:

(1) We accept full accountability for the accuracy and integrity of all content in our written and oral reports.
(2) We have examined and validated all AI-assisted contributions included in our written and oral reports against actual empirical runs on our database environment.
(3) We acknowledge the inherent limitations of GenAI systems, including the potential for technical errors, factual inaccuracies, hallucinations, or biases.
(4) We fully comprehend the AI-generated outputs used in our written and oral reports. We understand that the inability to explain these concepts in our own words during evaluations will be treated as a violation of academic integrity, and we accept the consequences of any such misuse.

D Record of Contribution

In accordance with the guidelines for this activity, Table 9 declares the individual contributions and sub-deliverables completed by each group member across all phases of the project.

Table 5: Appendix Table A1: Two-Column Interleaved Timelines for Dirty Read across All Four ANSI Isolation Levels

Time (t_x)	Session 1: Writer (`account_id = 101`)	Session 2: Reader (`account_id = 101`)
Configuration 1.1: READ UNCOMMITTED — Verdict: ANOMALY OBSERVED		
t = 0.0s	START TRANSACTION; UPDATE accounts SET balance = balance - 300.00 WHERE account_id = 101;	DO SLEEP(2);
t = 0.0s	SELECT 'W after update', balance → **700.00**; DO SLEEP(6);	(waiting on sleep)
t = 2.0s	(holding uncommitted update)	START TRANSACTION; SELECT 'R1', balance → **700.00 [DIRTY READ]**;
t = 6.0s	ROLLBACK; SELECT 'W after rollback', balance → **1000.00**;	DO SLEEP(6);
t = 8.0s	(transaction finished)	SELECT 'R2', balance → **1000.00**; COMMIT;
Configuration 1.2: READ COMMITTED — Verdict: PREVENTED		
t = 0.0s	START TRANSACTION; UPDATE accounts SET balance = balance - 300.00 WHERE account_id = 101;	DO SLEEP(2);
t = 0.0s	SELECT 'W after update', balance → **700.00**; DO SLEEP(6);	(waiting on sleep)
t = 2.0s	(holding uncommitted update)	START TRANSACTION; SELECT 'R1', balance → **1000.00 [NO DIRTY READ]**;
t = 6.0s	ROLLBACK; SELECT 'W after rollback', balance → **1000.00**;	DO SLEEP(6);
t = 8.0s	(transaction finished)	SELECT 'R2', balance → **1000.00**; COMMIT;
Configuration 1.3: REPEATABLE READ — Verdict: PREVENTED		
t = 0.0s	START TRANSACTION; UPDATE accounts SET balance = balance - 300.00 WHERE account_id = 101;	DO SLEEP(2);
t = 0.0s	SELECT 'W after update', balance → **700.00**; DO SLEEP(6);	(waiting on sleep)
t = 2.0s	(holding uncommitted update)	START TRANSACTION; SELECT 'R1', balance → **1000.00 [NO DIRTY READ]**;
t = 6.0s	ROLLBACK; SELECT 'W after rollback', balance → **1000.00**;	DO SLEEP(6);
t = 8.0s	(transaction finished)	SELECT 'R2', balance → **1000.00**; COMMIT;
Configuration 1.4: SERIALIZABLE — Verdict: PREVENTED VIA LOCK CONTENTION		
t = 0.0s	START TRANSACTION; UPDATE accounts SET balance = balance - 300.00 WHERE account_id = 101; [Holds X-Lock]	DO SLEEP(2);
t = 0.0s	SELECT 'W after update', balance → **700.00**; DO SLEEP(6);	(waiting on sleep)
t = 2.0s	(holding Exclusive X-Lock)	START TRANSACTION; SELECT 'R1', balance → **BLOCKED** (waits for S-Lock);
t = 6.0s	ROLLBACK; [Releases X-Lock]; SELECT 'W after rollback' → **1000.00**;	**UNBLOCKED** (waited 4.0s) → SELECT 'R1', balance → **1000.00**; DO SLEEP(6);
t = 12.0s	(transaction finished)	SELECT 'R2', balance → **1000.00**; COMMIT;

Table 6: Appendix Table A2: Two-Column Interleaved Timelines for Non-Repeatable Read across All Four ANSI Isolation Levels

Time (t_x)	Session 1: Reader (`account_id = 101`)	Session 2: Writer (`account_id = 101`)
Configuration 2.1: READ UNCOMMITTED — Verdict: ANOMALY OBSERVED		
t = 0.0s	START TRANSACTION; SELECT 'R1', balance → **1000.00**; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding transaction open)	START TRANSACTION; UPDATE accounts SET balance = balance + 200.00 WHERE account_id = 101; COMMIT;
t = 2.0s	(holding transaction open)	SELECT 'W committed', balance → **1200.00**;
t = 6.0s	SELECT 'R2', balance → **1200.00 [NON-REPEATABLE READ:** 1000 ≠ 1200**]**; COMMIT;	(transaction finished)
Configuration 2.2: READ COMMITTED — Verdict: ANOMALY OBSERVED		
t = 0.0s	START TRANSACTION; SELECT 'R1', balance → **1000.00**; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding transaction open)	START TRANSACTION; UPDATE accounts SET balance = balance + 200.00 WHERE account_id = 101; COMMIT;
t = 2.0s	(holding transaction open)	SELECT 'W committed', balance → **1200.00**;
t = 6.0s	SELECT 'R2', balance → **1200.00 [NON-REPEATABLE READ: fresh Read View]**; COMMIT;	(transaction finished)
Configuration 2.3: REPEATABLE READ — Verdict: PREVENTED		
t = 0.0s	START TRANSACTION; SELECT 'R1', balance → **1000.00** [Read View created]; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding transaction open)	START TRANSACTION; UPDATE accounts SET balance = balance + 200.00 WHERE account_id = 101; COMMIT;
t = 2.0s	(holding transaction open)	SELECT 'W committed', balance → **1200.00**;
t = 6.0s	SELECT 'R2', balance → **1000.00 [REPEATABLE READ: Read View reused]**; COMMIT;	(transaction finished)
Configuration 2.4: SERIALIZABLE — Verdict: PREVENTED VIA LOCK CONTENTION		
t = 0.0s	START TRANSACTION; SELECT 'R1', balance → **1000.00** [Acquires S-Lock]; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding Shared S-Lock)	START TRANSACTION; UPDATE accounts SET balance = balance + 200.00 → **BLOCKED**;
t = 6.0s	SELECT 'R2', balance → **1000.00**; COMMIT; [Releases S-Lock];	**UNBLOCKED** (waited 4.0s) → UPDATE completes; COMMIT;
t = 6.0s	(transaction finished)	SELECT 'W committed', balance → **1200.00**;

Table 7: Appendix Table A3: Two-Column Interleaved Timelines for Phantom Read across All Four ANSI Isolation Levels

Time (t_x)	Session 1: Reader (`WHERE balance >= 800.00`)	Session 2: Inserter (`account_id = 104`)
Configuration 3.1: READ UNCOMMITTED — Verdict: ANOMALY OBSERVED		
t = 0.0s	START TRANSACTION; SELECT 'P1', account_id → **[101, 103]** (2 rows); DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding transaction open)	START TRANSACTION; INSERT INTO accounts VALUES (104, 4, 1500.00, 'ACTIVE'); COMMIT;
t = 6.0s	SELECT 'P2', account_id → **[101, 103, 104]** (3 rows) **[PHANTOM READ]**;	(transaction finished)
t = 6.0s	UPDATE accounts SET status = 'FLAGGED' WHERE balance >= 800.00;	(transaction finished)
t = 6.0s	SELECT 'P3', account_id → **[101, 103, 104]**; COMMIT;	(transaction finished)
Configuration 3.2: READ COMMITTED — Verdict: ANOMALY OBSERVED		
t = 0.0s	START TRANSACTION; SELECT 'P1', account_id → **[101, 103]** (2 rows); DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding transaction open)	START TRANSACTION; INSERT INTO accounts VALUES (104, 4, 1500.00, 'ACTIVE'); COMMIT;
t = 6.0s	SELECT 'P2', account_id → **[101, 103, 104]** (3 rows) **[PHANTOM READ: fresh Read View]**;	(transaction finished)
t = 6.0s	UPDATE accounts SET status = 'FLAGGED' WHERE balance >= 800.00;	(transaction finished)
t = 6.0s	SELECT 'P3', account_id → **[101, 103, 104]**; COMMIT;	(transaction finished)
Configuration 3.3: REPEATABLE READ — Verdict: WRITE-ONLY PHANTOM OCCURS		
t = 0.0s	START TRANSACTION; SELECT 'P1', account_id → **[101, 103]** (2 rows) [Snapshot created]; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding transaction open)	START TRANSACTION; INSERT INTO accounts VALUES (104, 4, 1500.00, 'ACTIVE'); COMMIT;
t = 6.0s	SELECT 'P2', account_id → **[101, 103]** (2 rows) **[HIDDEN BY MVCC SNAPSHOT]**;	(transaction finished)
t = 6.0s	UPDATE accounts SET status = 'FLAGGED' WHERE balance >= 800.00; [Current-read updates row 104];	(transaction finished)
t = 6.0s	SELECT 'P3', account_id → **[101, 103, 104]** (3 rows) **[WRITE-ONLY PHANTOM REVEALED]**; COMMIT;	(transaction finished)
Configuration 3.4: SERIALIZABLE — Verdict: PREVENTED VIA NEXT-KEY GAP LOCK		
t = 0.0s	START TRANSACTION; SELECT 'P1', account_id → **[101, 103]** (2 rows) [Holds Next-Key Locks]; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding Next-Key Locks on range and gap)	START TRANSACTION; INSERT INTO accounts VALUES (104, 4, 1500.00, 'ACTIVE') → **BLOCKED**;
t = 6.0s	SELECT 'P2', account_id → **[101, 103]** (2 rows);	(waiting on Next-Key lock)
t = 6.0s	UPDATE accounts SET status = 'FLAGGED' WHERE balance >= 800.00; (affected = 2 rows);	(waiting on Next-Key lock)
t = 6.0s	SELECT 'P3', account_id → **[101, 103]** (2 rows); COMMIT; [Releases Next-Key Locks];	**UNBLOCKED** (waited 4.0s) → INSERT completes; COMMIT;

Table 8: Appendix Table B1: Two-Column Interleaved Timelines for the Explicit Locking Experiments

Time (t_x)	Session 1	Session 2
Lock Case 1: READ UNCOMMITTED + `FOR UPDATE` (Dirty Read) — Verdict: PREVENTED		
t = 0.0s	START TRANSACTION; UPDATE accounts SET balance = balance - 300.00 WHERE account_id = 101; [Holds X-Lock] → **700.00**; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding uncommitted update)	START TRANSACTION; SELECT 'R1' ... FOR UPDATE → **BLOCKED** (needs X-Lock);
t = 6.0s	ROLLBACK; [Releases X-Lock]; SELECT 'W after rollback' → **1000.00**;	**UNBLOCKED** (waited 4.0s) → 'R1' → **1000.00 [NO DIRTY READ]**; DO SLEEP(2);
t = 8.0s	(transaction finished)	SELECT 'R2' ... FOR UPDATE → **1000.00**; COMMIT;
Lock Case 2: READ COMMITTED + `FOR SHARE` (Non-Repeatable Read) — Verdict: PREVENTED		
t = 0.0s	START TRANSACTION; SELECT 'R1' ... FOR SHARE → **1000.00** [Holds S-Lock]; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding S-Lock)	START TRANSACTION; UPDATE accounts SET balance = balance + 200.00 WHERE account_id = 101 → **BLOCKED**;
t = 6.0s	SELECT 'R2' ... FOR SHARE → **1000.00 [REPEATABLE]**; COMMIT; [Releases S-Lock];	**UNBLOCKED** (waited 4.0s) → UPDATE completes; COMMIT; SELECT 'W committed' → **1200.00**;
Lock Case 3: REPEATABLE READ + Locking Range Read (Phantom Read) — Verdict: PREVENTED		
t = 0.0s	START TRANSACTION; SELECT 'P1' ... WHERE balance >= 800.00 FOR UPDATE → **[101, 103]** [Next-Key Locks on all rows and gaps]; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding Next-Key Locks)	START TRANSACTION; INSERT INTO accounts VALUES (104, 4, 1500.00, 'ACTIVE') → **BLOCKED** (insert-intention vs. gap lock);
t = 6.0s	SELECT 'P2' ... FOR UPDATE → **[101, 103] [NO PHANTOM]**; COMMIT; [Releases locks];	**UNBLOCKED** (waited 4.0s) → INSERT completes; COMMIT;
Lock Case 4: READ COMMITTED + `LOCK TABLES accounts READ` (Phantom Read) — Verdict: PREVENTED		
t = 0.0s	LOCK TABLES accounts READ; SELECT 'P1' → **[101, 103]**; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding table READ lock)	START TRANSACTION; INSERT INTO accounts VALUES (104, 4, 1500.00, 'ACTIVE') → **BLOCKED** (table lock);
t = 6.0s	SELECT 'P2' → **[101, 103] [NO PHANTOM]**; UNLOCK TABLES;	**UNBLOCKED** (waited 4.0s) → INSERT completes; COMMIT;
Lock Case 5: SERIALIZABLE + `FOR UPDATE` on Account 101; S2 writes Account 102 — Verdict: NO COLLATERAL WAIT		
t = 0.0s	START TRANSACTION; SELECT 'R1' ... FOR UPDATE → **1000.00** [X-Lock on 101 only]; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding X-Lock on 101)	START TRANSACTION; UPDATE accounts SET balance = balance + 50.00 WHERE account_id = 102 → **Completes Immediately (0.0s wait)**; COMMIT; SELECT 'W committed' → **550.00**;
t = 6.0s	SELECT 'R2' ... FOR UPDATE → **1000.00**; COMMIT;	(transaction finished)
Lock Case 6: SERIALIZABLE + `LOCK TABLES accounts WRITE`; S2 writes Account 102 — Verdict: COLLATERAL WAIT		
t = 0.0s	LOCK TABLES accounts WRITE; SELECT 'R1' → **1000.00**; DO SLEEP(6);	DO SLEEP(2);
t = 2.0s	(holding table WRITE lock)	START TRANSACTION; UPDATE accounts SET balance = balance + 50.00 WHERE account_id = 102 → **BLOCKED**;
t = 6.0s	SELECT 'R2' → **1000.00**; UNLOCK TABLES;	**UNBLOCKED** (waited 4.0s) → UPDATE completes; COMMIT; SELECT 'W committed' → **550.00**;

Table 9: Record of Group Member Contributions

Role / Phase	Claimed by	Task Breakdown & Sub-deliverables
1. Schema & Scenario Design	Airon Matthew Bantillo	Designed relational schema (`customer`, `accounts`, `transactions`), defined initial baseline data seeds, formulated the 3 concurrency anomaly scenarios (Dirty Read, Non-Repeatable Read, and Phantom Read), and created initial transaction timing diagrams.
2. DB Setup & Transaction Scripts	Bullet Andre Fernandez	Created automated DDL/DML setup and reset scripts (`00_schema.sql`, `01_seed.sql`, `02_reset.sql`), implemented dual-session SQL transaction script pairs with controlled timing (`DO SLEEP` offsets) for all test cases.
3. Baseline Isolation Experiments	Jean Rondel Ponce	Executed and validated all 12 baseline matrix test cases across the 4 ANSI SQL-92 isolation levels (**READ UNCOMMITTED**, **READ COMMITTED**, **REPEATABLE READ**, **SERIALIZABLE**) against MySQL InnoDB, captured timestamped event logs, and documented observed read anomalies.
4. Lock Experiments & Performance	Charles Sebastian Infante	Implemented and executed explicit locking mechanisms (`FOR UPDATE`, `FOR SHARE`, and `LOCK TABLES`), recorded lock blocking metrics, developed the concurrency benchmarking suite, and measured throughput and latency trade-offs.
5. Results, Report & Presentation	Alec Marx Gabriel Nono	Synthesized isolation level comparison matrices and timelines, compiled and formatted the technical report (LaTeX), drafted presentation materials, and coordinated live defense preparation.